GEMINI CLI:
A2 : Concrete Architecture

Group Name : Group 25

CISC 322/326

Date : March 13, 2025

Video Link : https://youtu.be/ObDranHxiVU

Group Names & Roles

Caravaggio, Emanuel - Presenter , Introduction & Overview, Conclusion, AI Collaboration , Reflexion Analysis (Student I.D : 20403106)

Clulow, Damon - Components and Interactions, Abstract, Naming Convention (Student I.D : 20285634)

Dhungana, Raghab - Architectural Styles, Reflexion Analysis (Student I.D : 20354596)

Elamin, Adam - Use Cases , Final Formatting (Student I.D : 20271708)

Fernando, Paul - Team Leader, Subsystem Architecture, Use Cases, Lessons Learnt (Student I.D : 20261647)

Shetty, Aditya - Presenter , Components and Interactions, Data Dictionary (Student I.D : 20320373)

Table Of Contents

Abstract - 4

System Overview - - - - - - - - - - - - - 5

Components and Interactions : Directory Structure - - - - - - - - - - - - 6

Directory Structure - - - - - - - - - - - - 7

Components and Interactions : Dependency Graph - - - - - - - - - - - - - 8

Dependency Graph - - - - - - - - - - - - - 9

Architecture Styles - 10

Concurrency - 11

Reflexion Analysis - 12

Top-Level Subsystem Analysis - - - - - - - - - - - - - - - - - 13

Conceptual Vs. Concrete Architecture - - - - - - - 14

Use Case #1 - 15

Use Case #2 - 16

Data Dictionary - 17

Lessons Learned - 18

Conclusion - 19

AI Collaboration - 20

Abstract

- In our Concrete Architecture we analyzed Gemini CLI's subsystems using the Understand tool

- We identified the Gemini CLI as a layered architecture style, with client-server interaction that communicated asynchronously through event driven communication with other subsystem components

- Our chose top-level subsystem was the Orchestration engine. We compared its conceptual role against concrete implementation via submodule decomposition

- Finally we described 2 use cases , project summarization and automated issue triage, described through sequence diagrams showing concrete method calls across components

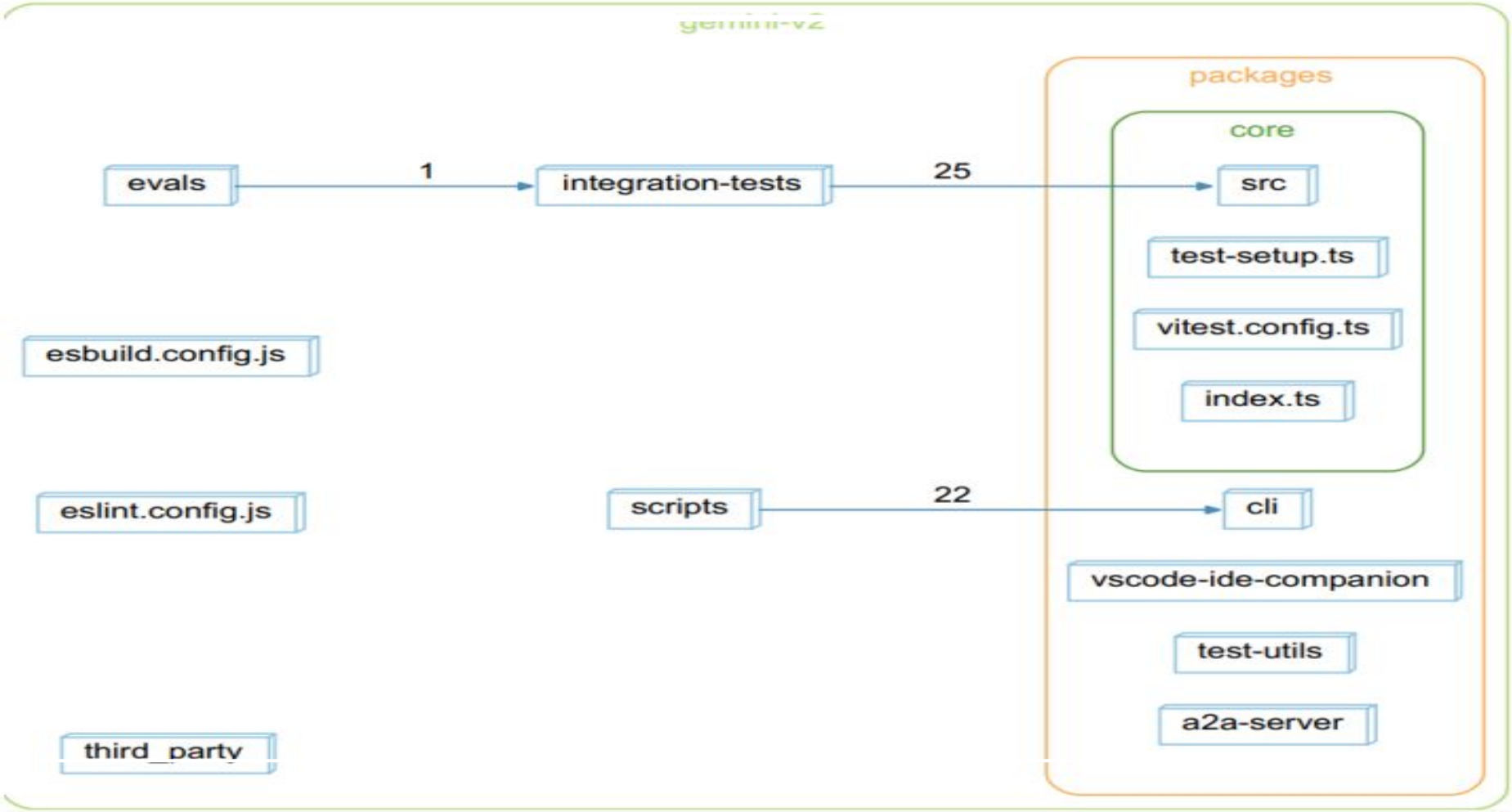
System Overview

- The system is organized primarily within the packages/ directory, where the CLI package serves as the presentation layer and Core implements the primary orchestration logic
- Using the Understand tool we found additional subsystems including tools, agents, configuration, models, and external integrations.
- In our conceptual architecture we describe these subsystems as functions of the Core and CLI, however they act more as dependent components that extend the system's core capabilities
- These components allow users to submit prompts, execute shell commands, perform web searches, and retrieve context using the CLI as the frontend interaction tool, and the Core and the backend processor

Components and Interactions : Directory Structure

- We used the Understand tool to generate a dependency graph by directory, which revealed all the major subsystems
- Core was obviously the most internally complex subsystem containing the src/ directory which houses all of its dependent subcomponents including tools, agents, scheduler, safety, and config.
- In our conceptual report these components were referenced as actions that the Core could implement rather than dependent subsystems.
- Scripts has 22 direct references to the CLI, and integration-tests has 25 to the Core's src/ directory, this reflects that the testing infrastructure is highly dependent on these two.

Understand Tool , Concrete Architecture Subsystem Architecture :



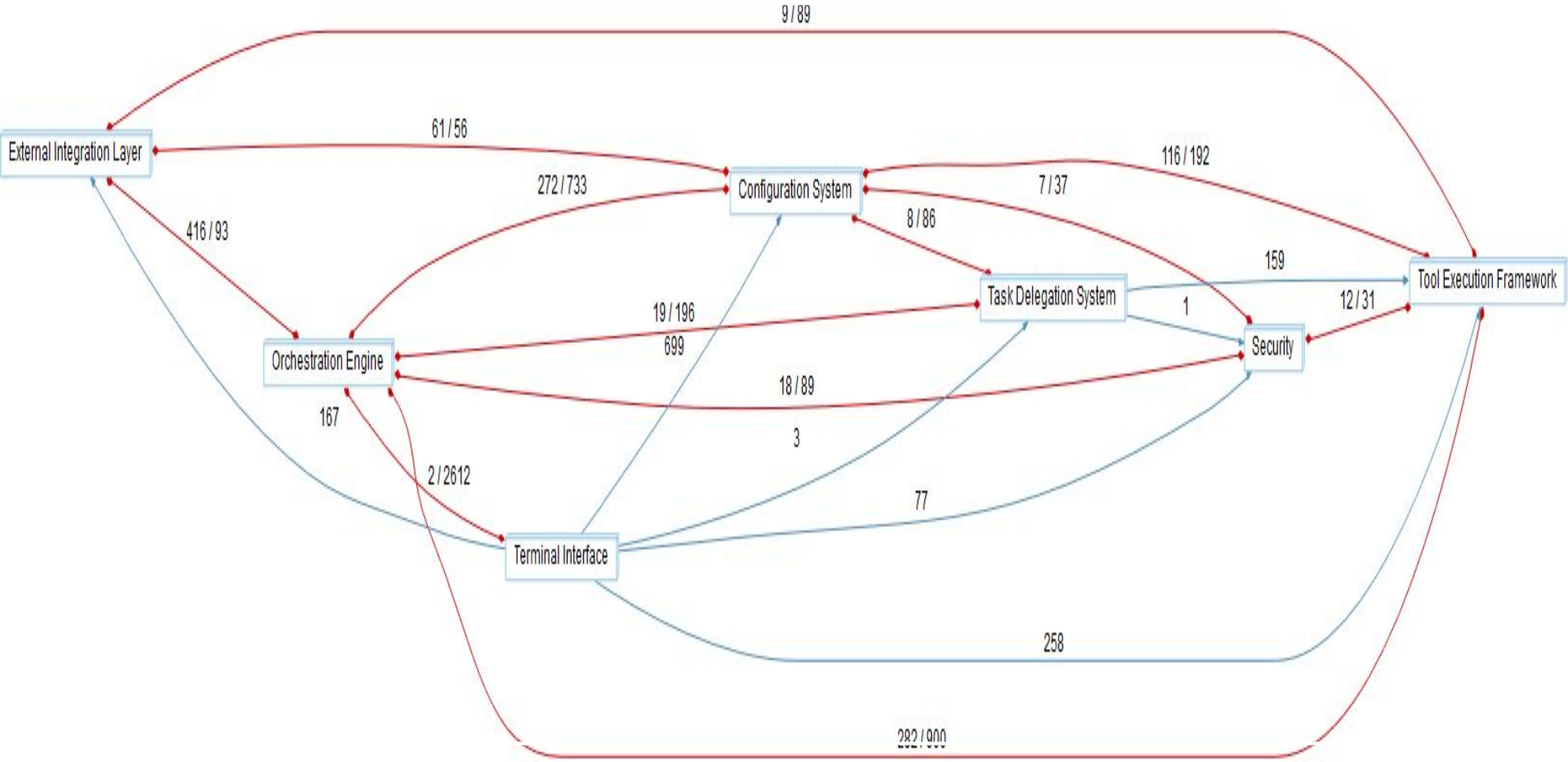
Components and Interactions : Dependency Graph

- Using the dependency graph we analyzed the Orchestration Engine. This subsystem includes the highest volume of dependencies, notably 2/2612 with the Terminal Interface and 272/733 with the Configuration System

- Intuitively this means that the terminal makes 2612 references into the Orchestration engine, while the latter makes 2 back. This describes a mostly one-way relationship.

- The Configuration System maintains two-way dependencies with nearly all top-level components, meaning it both configures and monitors system-wide logic

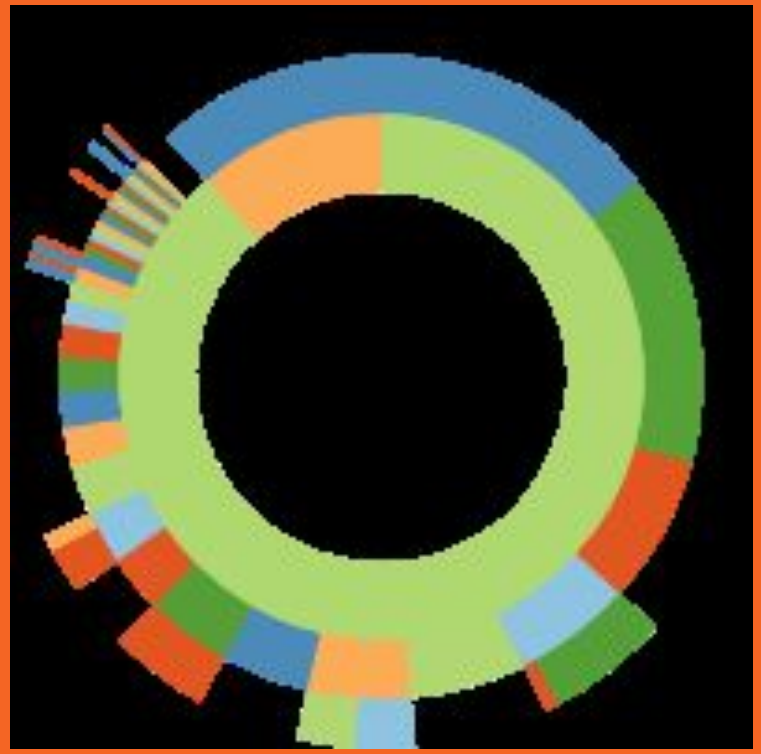
- The Tool Execution Framework is the most referenced leaf component, receiving 258 dependencies from Terminal Interface , Orchestration Engine, and Task Delegation System.


9 / 89
External Integration Layer
61 / 56
Configuration System
116 / 192
272 / 733
7 / 37
416 / 93
8 / 86
159
Task Delegation System
Tool Execution Framework
1
12 / 31
19 / 196
699
Security
Orchestration Engine
18 / 89
167
3
2 / 2612
77
Terminal Interface
258
282 / 900

Architecture Styles

- The concrete architecture style is a layered style with client-server components, this is unchanged from our conceptual architecture

- All major subsystems are organized under packages/, with each dependency working in descending order , this confirms a layered separation between the Core and CLI



Concurrency

- The structure describes an asynchronous event-driven concurrency model
- The CLI responds to user input while Core handles all latency operations
- Tool invocations are executed in the order they appear, meaning multiple requests cannot be run at the same time
- Individual tools like file searching and utilities like ripgrep run in parallel. This keeps performance levels high without violating the sequential invocation model.

Reflexion Analysis

High Level Analysis:

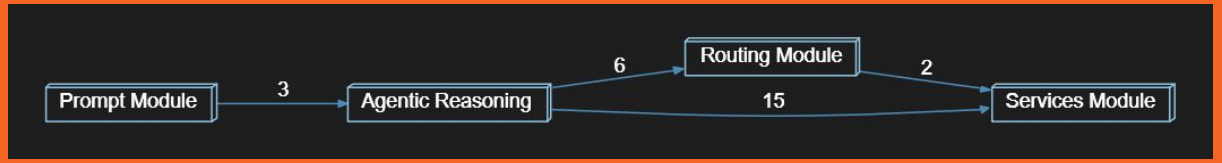
- Divergence in Component Hierarchy
- One-Way Dependency
- Exposure of Task Delegation system

2nd Level Analysis:

- Tool Execution vs. Tool Scheduling
- Security Infrastructure

Top-Level Subsystem Analysis: The Orchestration Engine

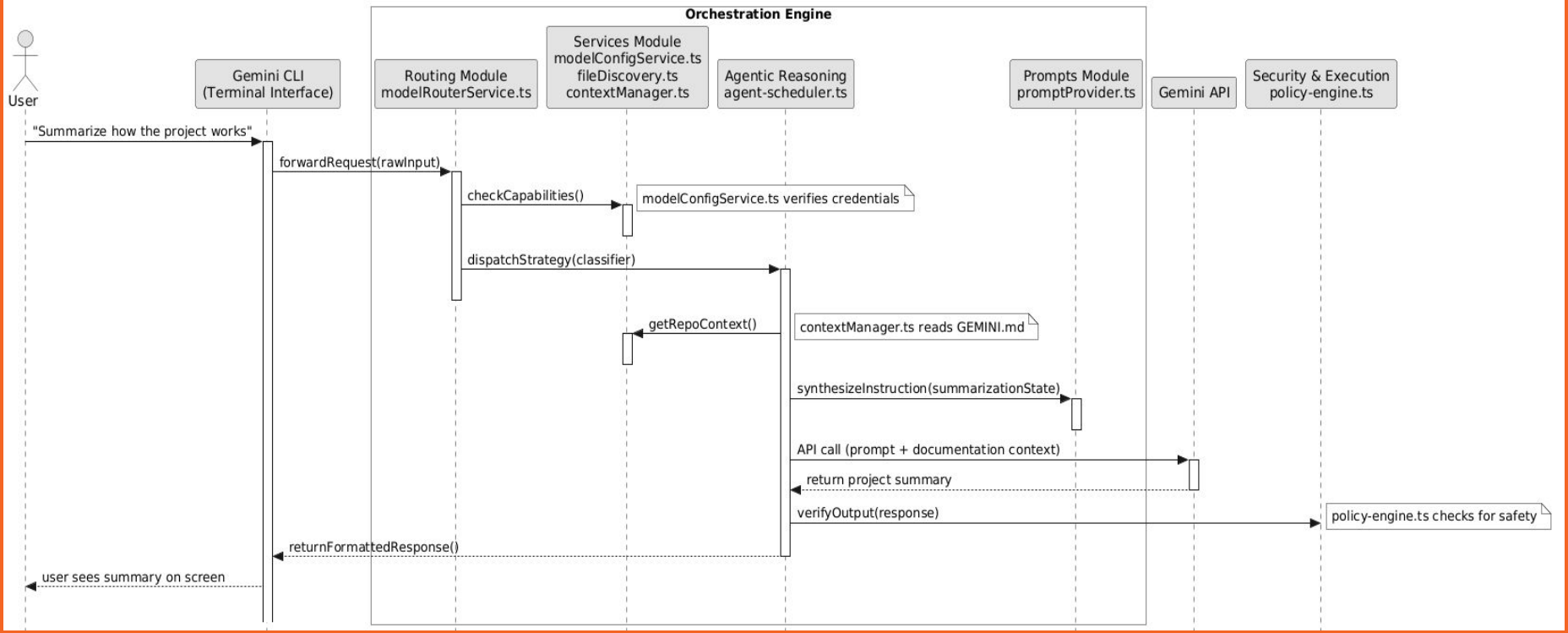
- Central brain of Gemini CLI
- Responsible for deconstructing high-level user prompts to actionable plans
- 4 subcomponents:
  - Routing module
  - Prompts module
  - Agentic Reasoning
  - Services module



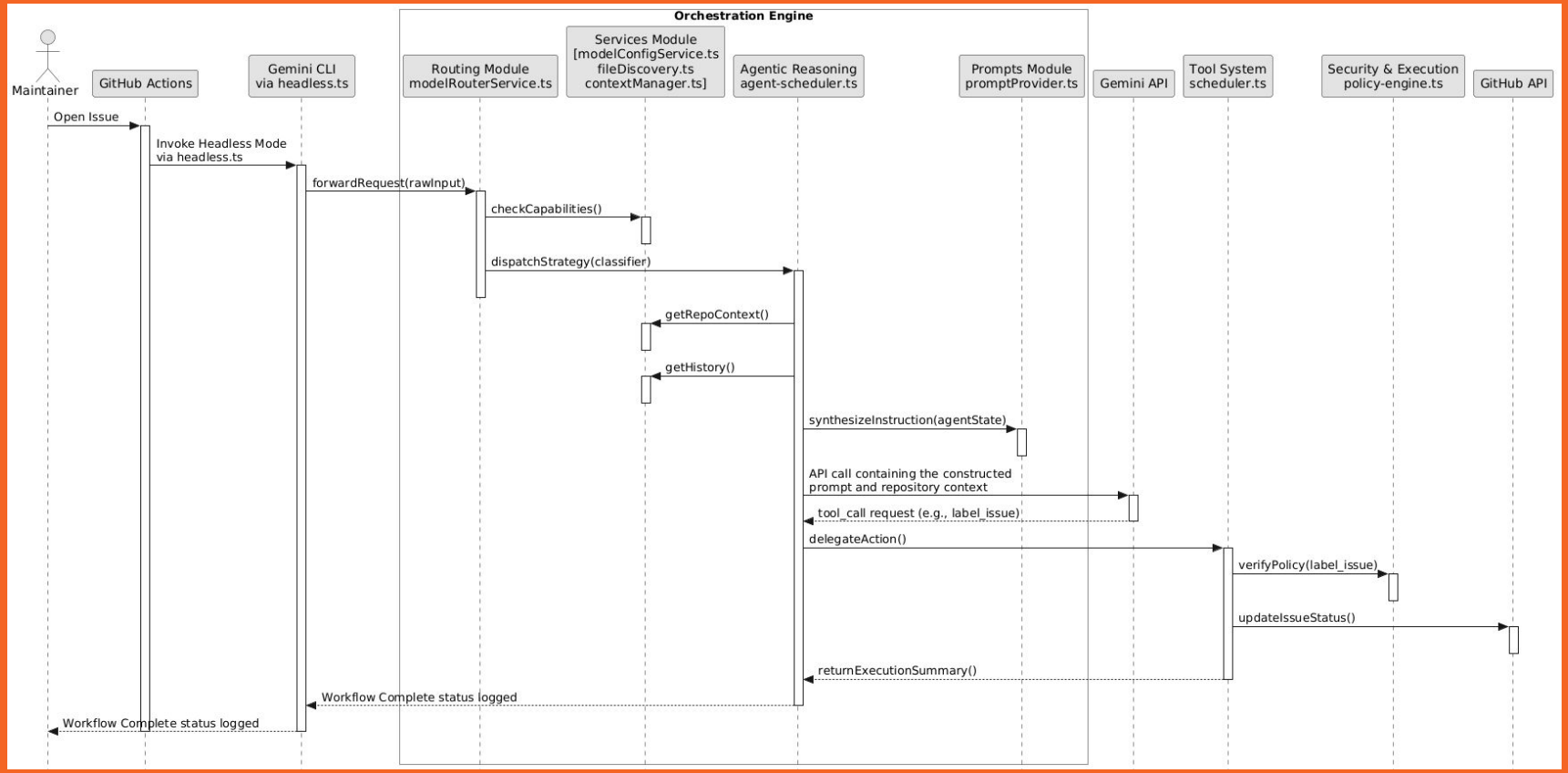
Orchestration Engine
Conceptual Vs. Concrete Architecture

- conceptual architecture, the Orchestration engine was represented as a single, monolithic component within the Application Layer. It was tasked with general “backend orchestration” and “API communication”.
- The actual implementation is far more granular, comprising of the aforementioned four sub-components.
- A significant divergence is the presence of the Routing Module . The conceptual model did not account for the complex intent classification and strategy selection logic required to handle multi-model environments.

Use Case #1


Orchestration Engine
User
Gemini CLI
(Terminal Interface)
Routing Module
modelRouterService.ts
Services Module
modelConfigService.ts
fileDiscovery.ts
contextManager.ts
Agentic Reasoning
agent-scheduler.ts
Prompts Module
promptProvider.ts
Gemini API
Security & Execution
policy-engine.ts
"Summarize how the project works"
forwardRequest(rawInput)
checkCapabilities()
modelConfigService.ts verifies credentials
dispatchStrategy(classifier)
getRepoContext()
contextManager.ts reads GEMINI.md
synthesizeInstruction(summarizationState)
API call (prompt + documentation context)
return project summary
verifyOutput(response)
policy-engine.ts checks for safety
returnFormattedResponse()
user sees summary on screen

Use Case #2


Orchestration Engine
Maintainer
GitHub Actions
Gemini CLI via headless.ts
Routing Module modelRouterService.ts
Services Module [modelConfigService.ts fileDiscovery.ts contextManager.ts]
Agentic Reasoning agent-scheduler.ts
Prompts Module promptProvider.ts
Gemini API
Tool System scheduler.ts
Security & Execution policy-engine.ts
GitHub API
Open Issue
Invoke Headless Mode via headless.ts
forwardRequest(rawInput)
checkCapabilities()
dispatchStrategy(classifier)
getRepoContext()
getHistory()
synthesizeInstruction(agentState)
API call containing the constructed prompt and repository context
tool_call request (e.g., label_issue)
delegateAction()
verifyPolicy(label_issue)
updateIssueStatus()
returnExecutionSummary()
Workflow Complete status logged
Workflow Complete status logged

Data Dictionary

Component	Type	Description
packages/cli/	Directory	Presentation Layer. Handles IO presentation and user configuration.
packages/core/	Directory	Backend Logic Layer. Manages most of the complex prompt disassembly and state management.
packages/core/src/tools	Directory	Manages the Tool Subsystem. Contains all memory for shell tools and MCP implementation.
packages/vscode-ide-companion	Directory	Extends VSCode Accessibility to the CLI.
packages/a2a-server/	Directory	Implements Agentic communication.

Lessons Learned

- Emphasized the need to analyze physical dependencies to truly understand system behaviour
- Reinforced the importance of continuous architectural reflection

Conclusions

- Concrete architecture supports determined layered structure
- CLI package functions as a presentation layer, Core package acts as a central orchestration component
- Divergences between conceptual and concrete model
- Orchestration subsystem and subcomponents

AI Collaboration

- Used to Identifying potential implementation responsibilities and verify assumptions made by group members
- focused on providing sufficient context and restricting the models responses to the information we supplied
- To ensure accuracy: only used Gemini CLI documentation, output was reviewed by group members, and cross-referenced with course materials
- Contributed 10% of our deliverable - acted as a small support tool